

Replikacja Eksperymentu 4.5 na Orange Pi Zero 3

Ten sam kod, inny procesor, inna architektura, inny system. Pytanie: czy wynik był własnością metody, czy konkretnej płytki?

Faza 1 — klasyfikator ciągły

✓ wynik identyczny z Raspberry Pi

RECALL

85, 0%

17 z 20 flag · RPi: 85%

PRECISION

100%

0 fałszywych alarmów

RAMKI

111 587

w 240 s · ~465/s

ZGUBIONE

0

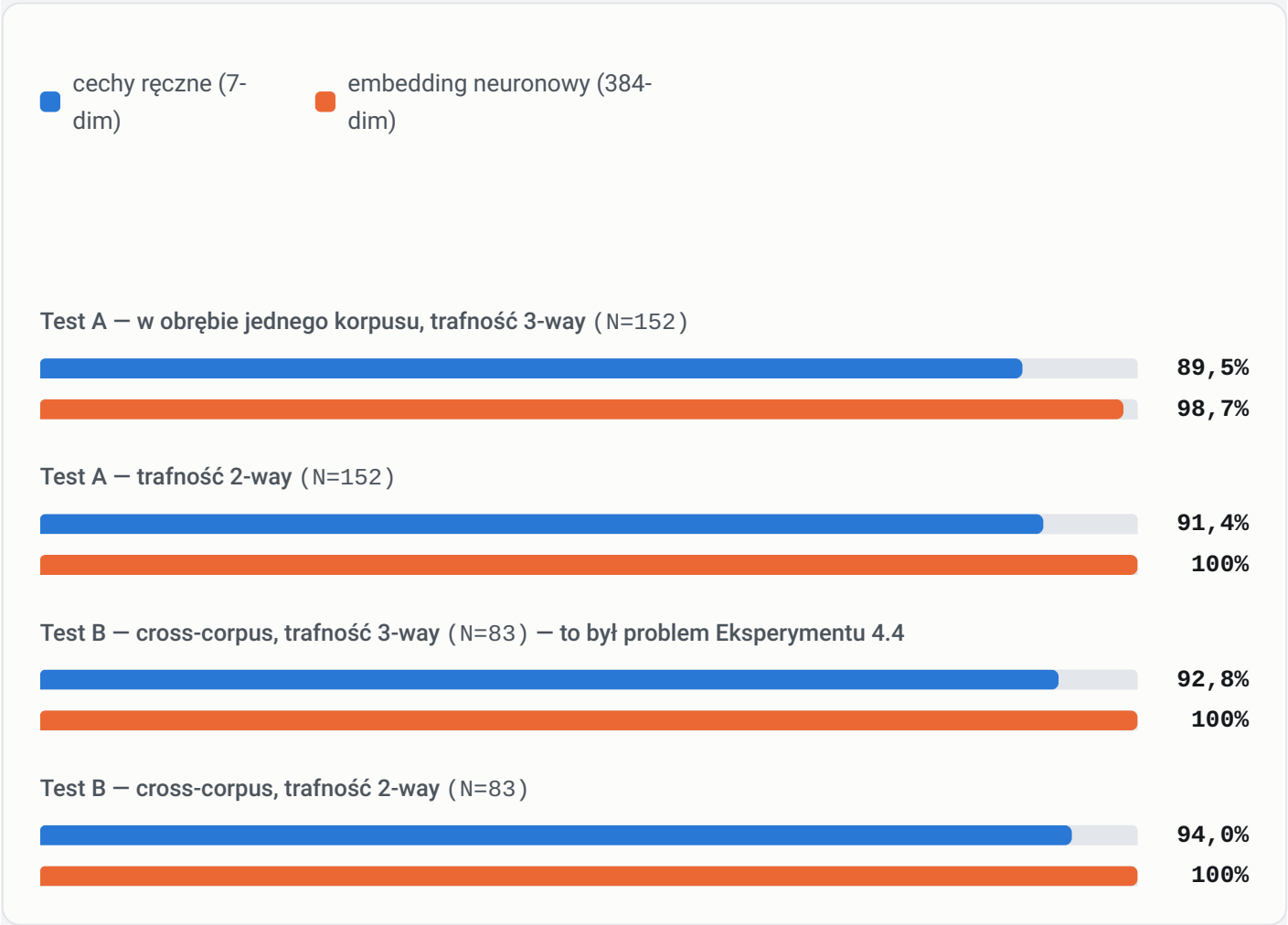
CPU ~13% jednego rdzenia

Skrypt `pi_continuous_observer.py` przeniesiony bez zmiany ani jednej linii. Przy wspólnym progu 0,5 zbiory wykrytych bajtów pokrywają się z Raspberry Pi w **11 z 12** pozycji — oba przebiegi używały programowo potwierdzonego, identycznego korpusu (seed 999).

Faza 5 — embeddingi neuronowe

pierwsze uruchomienie na sprzęcie docelowym

Na Raspberry Pi Zero W ten eksperyment był *niewykonalny* – PyTorch nie ma pakietów dla ARMv6. Orange Pi Zero 3 to aarch64, więc kod ruszył. Porównanie: ręcznie zaprojektowany wektor 7 cech kontra embedding neuronowy 384-wymiarowy (MiniLM).



Porównanie z przebiegiem na laptopie x86 (8 sierpnia 2026) – ta sama biblioteka i ten sam korpus testowy

TEST	METODA	ORANGE PI ZERO 3	LAPTOP X86	ZGODNOŚĆ
A (N=152)	ręczne 3-way	89,5%	89,5%	=
A (N=152)	neuronowe 3-way	98,7%	98,7%	=
A (N=152)	neuronowe 2-way	100%	100%	=
B (N=83)	ręczne 3-way	92,8%	92,8%	=
B (N=83)	ręczne 2-way	94,0%	94,0%	=
B (N=83)	neuronowe 3-way / 2-way	100%	100%	=

Całość liczyła się **2 min 26 s** (czas CPU 6 min 47 s – obliczenia realnie rozłożyły się na cztery rdzenie), przy zużyciu ~5% z 2 GB RAM.

`torch.cuda.is_available()` zwraca `False` – wszystko idzie przez CPU, GPU Mali nie bierze udziału.

Przebieg godzinny – pytanie o próg domknięte

✓ hipoteza potwierdzona

1 685 554 ramek w 3600 s, 0 zgubionych, obie fazy równolegle. Zamiast zgadywać z siatki progów, policzono dokładne współczynniki `big_jumps/changed_pairs` dla wszystkich 20 prawdziwych flag – urwisko leży precyzyjnie na **0,4595**.

Położenie „urwiska” progów w zależności od długości próby

PRZEBIEG	DŁUGOŚĆ	URWISKO
Orange Pi	240 s	~0,40
Orange Pi	1 h	0,4595
RPi Zero W (dokument strojenia)	1 h	0,46–0,47

Przy 15× dłuższej próbie urwisko przesunęło się z ~0,40 na 0,4595 – **w stronę wartości udokumentowanej i o jeden krok siatki od niej**. Wcześniejsza rozbieżność wynikała z niedokonwergowanych estymat, nie z platformy. Próg 0,3 pozostaje bezpieczny z zapasem.

Ustalenie kluczowe – to nie próg ogranicza wykrywalność

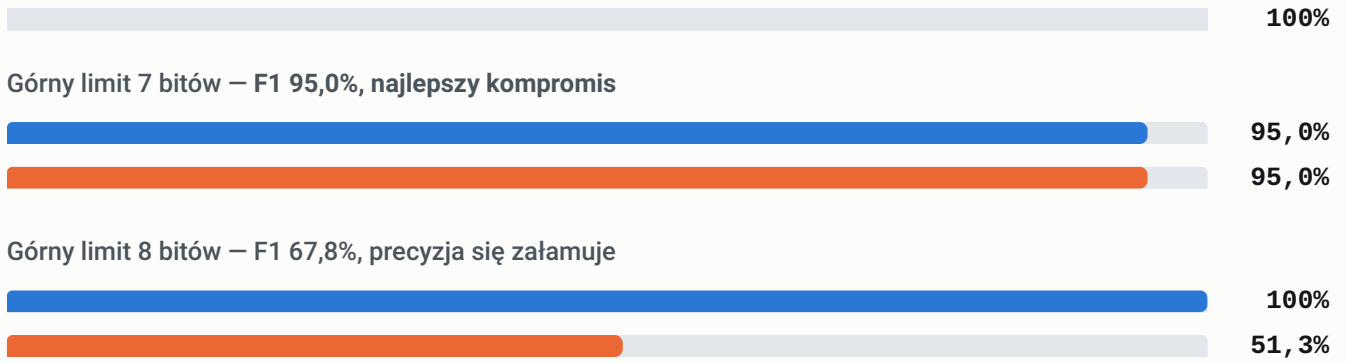
Analiza współczynników ujawniła coś, czego nie widać z samych metryk: **trzy przeoczone flagi nie są przeoczone przez próg**. Ich współczynniki to 0,97, 0,84 i 0,82 – przeszłyby każdy próg. Odrzuca je warunek `2 ≤ liczba bitów ≤ 6`, bo mają 7 lub 8 niezależnie przełączających się bitów.

■ Recall ■ Precision

Górny limit 6 bitów (obecny) – F1 91,9%



85,0%



Dlaczego limit 8 rujnuje precyzję — i czemu to ma sens

Bajt, w którym wszystkie osiem bitów przełącza się niezależnie, jest nieodróżnialny od szybko zmieniającego się skalar. Pierwotne ograniczenie było więc słuszne co do idei, tylko ustawione o **jeden krok za ciasno**. Podniesienie limitu do 7 kupuje +10 pp wykrywalności kosztem 5 pp precyzji — netto +3,1 pp F1. **Wymaga potwierdzenia na drugim korpusie** (inne ziarno) przed wdrożeniem — zmierzono to na jednym.

Faza 2 — embedded Qdrant, równolegle z Fazą

1

Ewaluacja leave-one-out na żywym ruchu, 11 pomiarów w ciągu godziny

METRYKA	ORANGE PI ZERO 3	RPI ZERO W (4.5)
trafność 3-way	94, 0%	91, 6%
trafność 2-way	96, 4%	91, 6%
N	83	83

Wynik nieco lepszy niż na Raspberry Pi, ale **nie należy tego nadinterpretować** — to pojedynczy przebieg, a w ciągu godziny wartość 3-way oscylowała między 91,6% a 95,2%. Istotne jest co innego: obie fazy działały jednocześnie, a płytka nie zgubiła ani jednej ramki.

Koszt integracji — obawa z analizy z 27.07 potwierdzona

Na Raspberry Pi uruchomienie MCP2515 to jedna linijka w `config.txt`. Tutaj trzeba było napisać własny overlay device-tree. Trzy konkretne pułapki:

1. **Na złączu wyprowadzony jest CS1, nie CS0.** PH5 (CS0) siedzi na pinie 3 jako I2C — na złącze trafia tylko PH9. W overlayu musi być `reg = <1>`.
2. **Standardowe overlaye Armbiana są niekompletne.** Bazowy węzeł SPI1 nie ma żadnego `pinctrl`, dostarczone `spidev*.dtbo` też go nie dodają — kontroler się rejestruje, ale piny milczą.
3. **SPI0 zajęty przez wbudowaną pamięć flash** i niedostępny na złączu — odwrotnie niż na Raspberry Pi, gdzie HAT-y siedzą na SPI0.

Dokumentacja publiczna pinoutu jest sprzeczna

Źródła w sieci podają różne pozycje fizyczne dla SPI. Rozstrzygnięto oficjalnym manuałem producenta (v1.1, str. 123) i weryfikacją krzyżową z device tree działającej płytki. Rację miał układ zgodny z Raspberry Pi: piny 19/21/23/24 plus przerwanie na 22.

Wniosek

Wynik Eksperymentu 4.5 jest własnością metody, nie platformy sprzętowej. Ten sam kod na innym SoC (Allwinner zamiast Broadcom), innej architekturze (aarch64 zamiast ARMv6), innym systemie (Armbian zamiast Raspberry Pi OS) i innej magistrali SPI dał identyczny wynik liczbowy. Dodatkowo Faza 5 — dotąd możliwa wyłącznie na laptopie — okazała się w pełni wdrażalna na sprzęcie docelowym, z wynikiem zgodnym co do ostatniej cyfry.

Uczciwa uwaga o czystości godzinnego przebiegu

Licznik `can0` zarejestrował **1 błąd RX** przy 1,83 mln ramek (`dropped 0`, `missed 0`). Przyczyna jest znana i leży po stronie prowadzącego pomiar: w trakcie trwającego eksperymentu wykonano `tailscale up --reset`, co chwilowo zresetowało konfigurację sieciową płytki. Sama instalacja pakietu, z obniżonym priorytetem CPU i I/O, przeszła bez śladu — licznik przed i po wynosił zero. Metryka kluczowa (`dropped`) pozostała zerowa, więc wynik jest ważny, ale przebieg nie był w 100% nieskażony i tak zostaje odnotowany.

Raport źródłowy: `Eksperyment_4.6_Replikacja_OrangePiZero3_20260814.md`

Ruch CAN: generator z Eksperymentu 4.3, seed 999, magistrala 250 kbit/s wspólna z PEAK PCAN-USB. Fazy 3 i 4 (wywołania płatnych API LLM) świadomie nie uruchamiane.